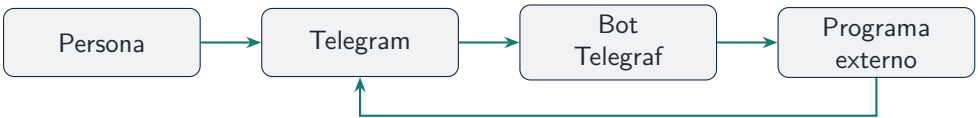
Un bot que recibe mensajes y ejecuta programas

Taller práctico con Node.js y Telegraf

Nelson · Noviembre 2026

La idea en una línea



Qué veremos

Cómo el bot recibe un mensaje, decide qué hacer, ejecuta un programa y devuelve el resultado.

El ejecutable de la maestría se presenta solo como una caja negra.

El bot de maestría: qué hace

- Guía a la persona por una sesión de optimización.
- Presenta soluciones y recibe decisiones con botones Sí/No.
- Coordina servicios y ejecutables; no hace el cálculo por sí mismo.
- Al final puede entregar resultados y una gráfica.

El recorrido

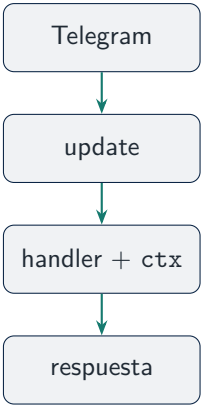
Formato → soluciones → decisiones → resultado final.

Telegraf recibe el mensaje

- Telegraf conecta Node.js con la API de Telegram.
- Un handler escucha un comando, un texto, una foto o un botón.
- `ctx` reúne el evento y herramientas para responder.

En este taller

`bot.launch()` usa long polling: el bot pregunta a Telegram si hay mensajes nuevos.



Un comando: validar, ejecutar, responder

```
bot.command('sumar', async (ctx) => {
  const numeros = leerDosNumeros(ctx.payload);
  if (!numeros) {
    return ctx.reply('Uso: /sumar 2 3');
  }

  try {
    const { stdout } = await proc.ejecutarPython(
      'sumar', numeros[0], numeros[1]
    );
    await ctx.reply(stdout);
  } catch (error) {
    await ctx.reply('No pude completar la suma.');
  }
});
```

Botones y continuidad de la sesión

Al pulsar Sí o No

Telegram envía una `callback_query`.
Telegraf la recibe con `bot.action(...)`.

El handler debe

Responder al botón con `answerCbQuery()`, guardar la elección y avanzar la sesión.

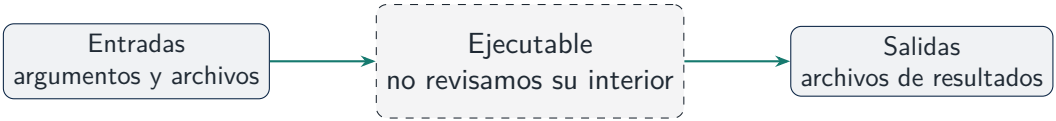
```
bot.action('SIV', async (ctx) => { ... })
```

Ejecutar un programa sin abrir un shell

```
execFile(programa, ['sumar', '2', '3'], {
  cwd: carpetaDelProyecto,
  timeout: 5000,
}, (error, stdout, stderr) => {
  // revisar error; usar stdout si todo salio bien
});
```

- `execFile` recibe el programa y sus argumentos por separado.
- No se debe concatenar texto de Telegram en un comando de `exec`.
- El proceso hereda la cuenta y los permisos de Windows del bot.
- Para el taller, Node ejecuta el ‘.exe‘ o inicia Python con el ‘.py‘ como argumento.

El ejecutable de la maestría: caja negra



Al bot le importa el contrato

Qué datos necesita el programa, dónde deja sus resultados y cómo informa un error.

Recibir un documento o una foto

1. Telegram entrega un `file_id` y algunos datos del archivo.
2. El bot pide un enlace temporal con `getFileLink()`.
3. Node descarga el contenido y lo guarda en `datos/descargas`.

```
const info = await descargar(
  ctx, documento.file_id, documento.file_name,
  documento.file_size
);
```

El ejemplo limita los archivos a 10 MB, limpia el nombre y no ejecuta lo que recibe.

Devolver un archivo

```
await ctx.replyWithDocument({
  source: './datos/reporte.pdf',
});

await ctx.replyWithPhoto({
  source: './datos/grafica.png',
});
```

- `ctx.reply(...)` envía texto al chat actual.
- `replyWithDocument` y `replyWithPhoto` suben archivos desde disco.

Permisos y seguridad: lo esencial

- Ejecuta el bot con una cuenta normal, no como administrador.
- Guarda el token en ‘.env‘; no lo publiques ni lo pongas en el código.
- Acepta solo operaciones conocidas y valida los argumentos.
- Usa `execFile`, fija la carpeta de trabajo y establece un tiempo límite.
- Limita y sanea los archivos que recibes.

Regla fácil de recordar

Un archivo recibido es un dato, no un programa que el bot deba ejecutar.

Práctica: probar el ejemplo

1. Copia ‘.env.example‘ a ‘.env‘ y agrega tu token de BotFather.
2. Ejecuta `npm install` y luego `npm run demo`.
3. Inicia el bot con `npm start` y prueba ‘/sumar 2 3‘.
4. Prueba ‘/exe sumar 7 5‘, ‘/demo‘ y envía un archivo pequeño.

Qué observar

La misma función de procesos se usa desde la demo de consola y desde el bot.

Usa un token de prueba: solo una instancia puede hacer polling con ese token.

Resumen

1. Telegraf recibe eventos y los entrega a handlers con `ctx`.
2. El handler valida, ejecuta la tarea y responde.
3. `execFile` separa el programa de sus argumentos.
4. Los archivos se descargan, se limitan y se guardan; nunca se ejecutan.
5. Del ejecutable de la maestría solo necesitamos conocer su entrada y su salida.

¿Preguntas?